

23rd Jan 26 | DA6401

→ Perceptron Innovations

start randomly → weighted inputs
↓
then → learning rule

continue to iteratively & backpropagate the loss, & improve.

now machine can automatically learn to classify the patterns

acted as the "foundaⁿ" for all future & modern DL!

$$\left. \begin{array}{l} z \geq 0 \Rightarrow 1 \\ z < 0 \Rightarrow 0 \end{array} \right\} \begin{array}{l} \text{MP} \\ \text{Neuron} \end{array}$$

- by adjusting the weights (w) & bias (b), the same architecture performs diff. tasks.
- The eqⁿ $w \cdot x + b = 0$ defines a line (in 2D) & hyperplane (n D).

$$w \cdot x + b > 0 \Rightarrow C_1$$

$$w \cdot x + b < 0 \Rightarrow C_2$$

w is always $\perp$ (orthogonal) to the decision boundary. Always pts. in "direcⁿ" of " C_1 " direcⁿ".
⇒ works very well for linearly separable

- if we Δ the ' w ' → orientaⁿ Δ s. Rotates the boundary around the center. It will Δ feature shift.
- if we Δ the ' b ' → position Δ s. It shifts the boundary w/o rotating it. Δ s the threshold eqⁿ. to fire.
- why bias? w/o b , $w \cdot x = 0$, it crosses origin. Not always, data is centered around origin. b acts as an offset.
↳ internal disposiⁿ. How easy or hard it is to trigger you, regardless of evidence

Evidence + Bias > 0 $\Rightarrow$ Action

Learning Problem

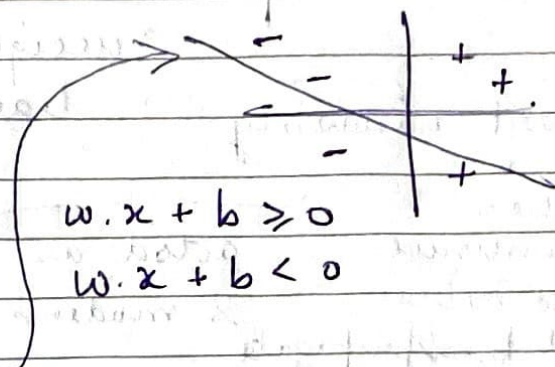
Goal: w, b s.t

+ samples

- samples

$$w \cdot x + b \geq 0$$

$$w \cdot x + b < 0$$



$\rightarrow$ we start with random wts

$\rightarrow$ use iterative error correction $\rightarrow$ loop through data,
for every mistake, nudge the wts.

Perceptron Learning

Input $D = \{(x, y)\}$

Init. $w \leftarrow 0$

Loop until convergence

1. $\hat{y} = \text{sign}(w \cdot x)$

2. $\hat{y} \neq y$

$$w \leftarrow w + x \quad (y=1, \text{ but got } -)$$

$$w \leftarrow w - x \quad (y=0, \text{ but got } +)$$

we have
to fix
the
angle.

too far away $\Rightarrow$ pull it (+)

too close $\Rightarrow$ push it (-)

- Pull

$$x_i \rightarrow y=1$$

$$\text{but } \hat{y}=0$$

$$(w \cdot x < 0) \rightarrow \text{we}$$

angle is $> 90^\circ$, so pull it

- Push

$x_i \Rightarrow y=0$
 $\hat{y}=1$, too close $< 90^\circ$
so push it.

- $\sum w_i x_i + b = 0 \Rightarrow$ linear hyperplane

it cannot bend or curve.

but, XOR can't be separated by perceptron as it is not linearly separable.

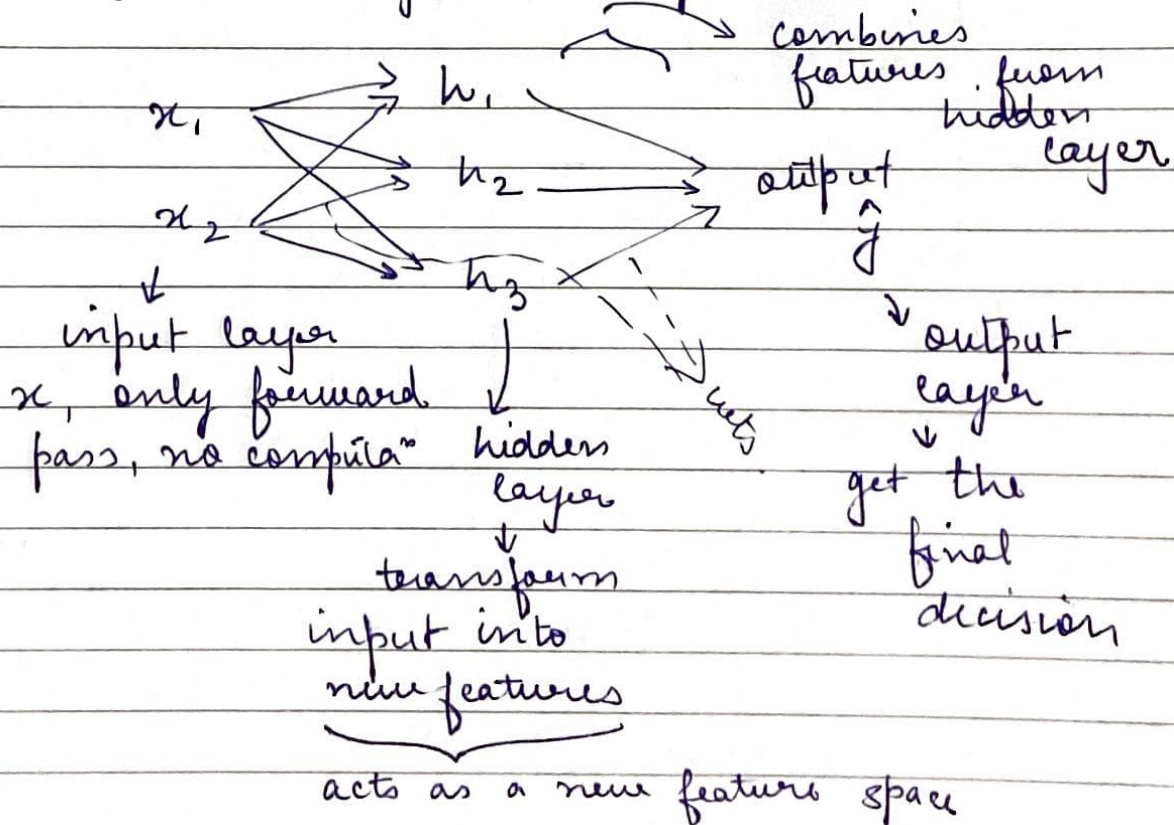
- Perceptron

Perceptron $\rightarrow$ it learns, has feature input, guaranteed convergences (if linearly sep.)

Cons $\rightarrow$ only linear separable works, has harsh threshold.

can't do gradient-based training.

$\Rightarrow$ MLP (Multi-layered Perceptron)



for h_1 $\leftarrow z_1 = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1$
for h_2 $\leftarrow z_2 = w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2$

- the network draws multiple lines & that helps to create a non-linear decision boundary. Hidden layers transform the space s.t. problem becomes linearly separable for the output layer.
- now we have solved the XOR problem